

Quickstart

Create a project and virtual environment

You'll only need to do this once.

```
mkdir my_project
cd my_project
python -m venv .venv
```

Activate the virtual environment

Do this every time you start a new terminal session.

```
source .venv/bin/activate
```

Install the Agents SDK

```
pip install openai-agents # or `uv add openai-agents`, etc
```

Set an OpenAI API key

If you don't have one, follow [these instructions](#) to create an OpenAI API key.

```
export OPENAI_API_KEY=sk-...
```

Create your first agent

Agents are defined with instructions, a name, and optional configuration such as a specific model.

```
from agents import Agent

agent = Agent(
    name="History Tutor",
    instructions="You answer history questions clearly and concisely.",
)
```

Run your first agent

Use `Runner` to execute the agent and get a `RunResult` back.

```
import asyncio
from agents import Agent, Runner

agent = Agent(
    name="History Tutor",
    instructions="You answer history questions clearly and concisely.",
)

async def main():
    result = await Runner.run(agent, "When did the Roman Empire fall?")
    print(result.final_output)

if __name__ == "__main__":
    asyncio.run(main())
```

For a second turn, you can either pass `result.to_input_list()` back into `Runner.run(...)`, attach a `session`, or reuse OpenAI server-managed state with `conversation_id` / `previous_response_id`. The [running agents](#) guide compares these approaches.

Use this rule of thumb:

If you want...	Start with...
Full manual control and provider-agnostic history	<code>result.to_input_list()</code>
The SDK to load and save history for you	<code>session=...</code>
OpenAI-managed server-side continuation	<code>previous_response_id</code> or <code>conversation_id</code>

For the tradeoffs and exact behaviors, see [Running agents](#).

Give your agent tools

You can give an agent tools to look up information or perform actions.

```
import asyncio
from agents import Agent, Runner, function_tool
```

```
@function_tool
def history_fun_fact() -> str:
    """Return a short history fact."""
    return "Sharks are older than trees."

agent = Agent(
    name="History Tutor",
    instructions="Answer history questions clearly. Use history_fun_fact when it helps.",
    tools=[history_fun_fact],
)

async def main():
    result = await Runner.run(
        agent,
        "Tell me something surprising about ancient life on Earth.",
    )
    print(result.final_output)

if __name__ == "__main__":
    asyncio.run(main())
```

Add a few more agents

Before you choose a multi-agent pattern, decide who should own the final answer:

- **Handoffs:** a specialist takes over the conversation for that part of the turn.
- **Agents as tools:** an orchestrator stays in control and calls specialists as tools.

This quickstart continues with **handoffs** because it is the shortest first example. For the manager-style pattern, see [Agent orchestration](#) and [Tools: agents as tools](#).

Additional agents can be defined in the same way. `handoff_description` gives the routing agent extra context about when to delegate.

```
from agents import Agent

history_tutor_agent = Agent(
    name="History Tutor",
    handoff_description="Specialist agent for historical questions",
    instructions="You answer history questions clearly and concisely.",
)

math_tutor_agent = Agent(
```

```
name="Math Tutor",
handoff_description="Specialist agent for math questions",
instructions="You explain math step by step and include worked examples.",
)
```

Define your handoffs

On an agent, you can define an inventory of outgoing handoff options that it can choose from while solving the task.

```
triage_agent = Agent(
    name="Triage Agent",
    instructions="Route each homework question to the right specialist.",
    handoffs=[history_tutor_agent, math_tutor_agent],
)
```

Run the agent orchestration

The runner handles executing individual agents, any handoffs, and any tool calls.

```
import asyncio
from agents import Runner

async def main():
    result = await Runner.run(
        triage_agent,
        "Who was the first president of the United States?",
    )
    print(result.final_output)
    print(f"Answered by: {result.last_agent.name}")

if __name__ == "__main__":
    asyncio.run(main())
```

Reference examples

The repository includes full scripts for the same core patterns:

- [examples/basic/hello_world.py](#) for the first run.
- [examples/basic/tools.py](#) for function tools.

- [examples/agent_patterns/routing.py](#) for multi-agent routing.

View your traces

To review what happened during your agent run, navigate to the [Trace viewer in the OpenAI Dashboard](#) to view traces of your agent runs.

Next steps

Learn how to build more complex agentic flows:

- Learn about how to configure [Agents](#).
- Learn about [running agents](#) and [sessions](#).
- Learn about [tools](#), [guardrails](#) and [models](#).